Task 05 - How a Multi-Agent System Is Built

Handbook reference: Page 5 of 10 | Report date: 2026-10-13 | Folder: `task-05-multi-agent-systems/`

1. Concept from the handbook

A multi-agent system has several agent instances with defined responsibilities and communication rules - possibly the same model. More agents are not automatically faster, cheaper or more accurate. Implementation and test planning can run in parallel when inputs are stable; review depends on the final diff. Two agents editing the same file conflict.

2. What was built

A **Delegation Contract** (Task, Inputs, Allowed files, Constraints, Output, Verify, Stop), a **task graph** with topological levels (parallel where possible), **file-conflict detection** that forces sequential execution, **scope enforcement** (an agent touching files outside its contract is rejected) and a **should_delegate** decision table.

3. Behaviours demonstrated

- Coordinator -> (Implementation || Test) -> Review
- Review always runs after implementation output exists
- Overlapping allowed files -> run sequentially
- Out-of-contract edits raise PermissionError
- Reviewer output notes it shares assumptions: a second AI is not proof

4. Files in this task

`demo.py`, `multi_agent.py`, `test_multi_agent.py`

Run it: `python demo.py` Test it: `python -m unittest discover`

5. Code screenshots

multi_agent.py

```python
"""Task 05 - Multi-agent system with a delegation contract and task graph.

Agents here are simulated roles (plain functions) so the coordination logic is
visible and testable. The contract, conflict detection, scheduling and
file-scope enforcement are real.
"""
from __future__ import annotations

from concurrent.futures import ThreadPoolExecutor
from dataclasses import dataclass, field
from typing import Callable, Dict, List


@dataclass
class Contract:
    """Delegation contract: Task, Inputs, Allowed files, Constraints, Output, Verify, Stop."""
    task: str
    inputs: List[str]
    allowed_files: List[str]
    constraints: List[str]
    output: str
    verify: str
    stop: str

    def validate(self) -> List[str]:
        # allowed_files may be empty on purpose: a read-only agent edits nothing
        return [k for k, v in self.__dict__.items() if not v and k != "allowed_files"]


@dataclass
class Node:
    role: str
    contract: Contract
    agent: Callable[[Contract, Dict[str, dict]], dict]    # returns {"touched": [...], "notes": str, ...}
    depends_on: List[str] = field(default_factory=list)


def should_delegate(files_touched: int, clean_boundaries: bool, needs_final_diff: bool) -> str:
    """Handbook table: 'Kab delegate karein?'"""
    if files_touched <= 1 and not clean_boundaries:
        return "ONE AGENT - avoid coordination overhead"
    if needs_final_diff:
        return "SEQUENTIAL - implementation -> review"
    if clean_boundaries:
        return "DELEGATE - feature + tests + review with clean boundaries"
    return "ONE AGENT - boundaries unclear"

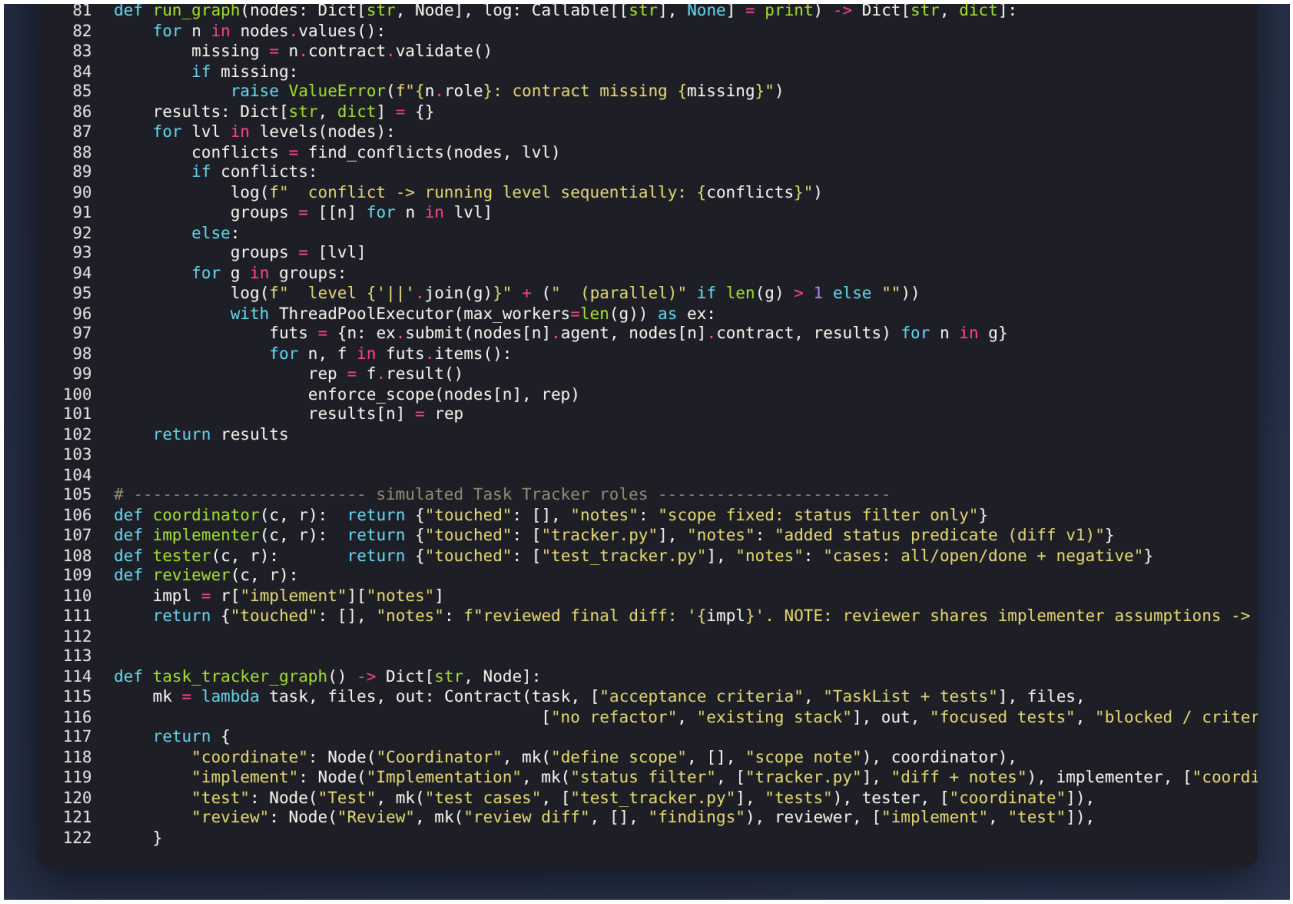

def find_conflicts(nodes: Dict[str, Node], parallel_group: List[str]) -> List[str]:
    """Two agents that may edit the same file must NOT run in parallel."""
    out, seen = [], {}
    for name in parallel_group:
        for f in nodes[name].contract.allowed_files:
            if f in seen:
                out.append(f"{f}: both '{seen[f]}' and '{name}' may edit it")
            seen[f] = name
    return out


def levels(nodes: Dict[str, Node]) -> List[List[str]]:
    """Topological levels: nodes in the same level can run in parallel."""
    done, result = set(), []
    remaining = dict(nodes)
    while remaining:
        ready = [n for n, v in remaining.items() if all(d in done for d in v.depends_on)]
        if not ready:
            raise ValueError("dependency cycle")
        result.append(sorted(ready))
        done.update(ready)
        for n in ready:
            remaining.pop(n)
    return result


def enforce_scope(node: Node, report: dict) -> None:
    bad = [f for f in report.get("touched", []) if f not in node.contract.allowed_files]
    if bad:
        raise PermissionError(f"agent '{node.role}' touched files outside contract: {bad}")


```

Main implementation: multi_agent.py (part 1/2)

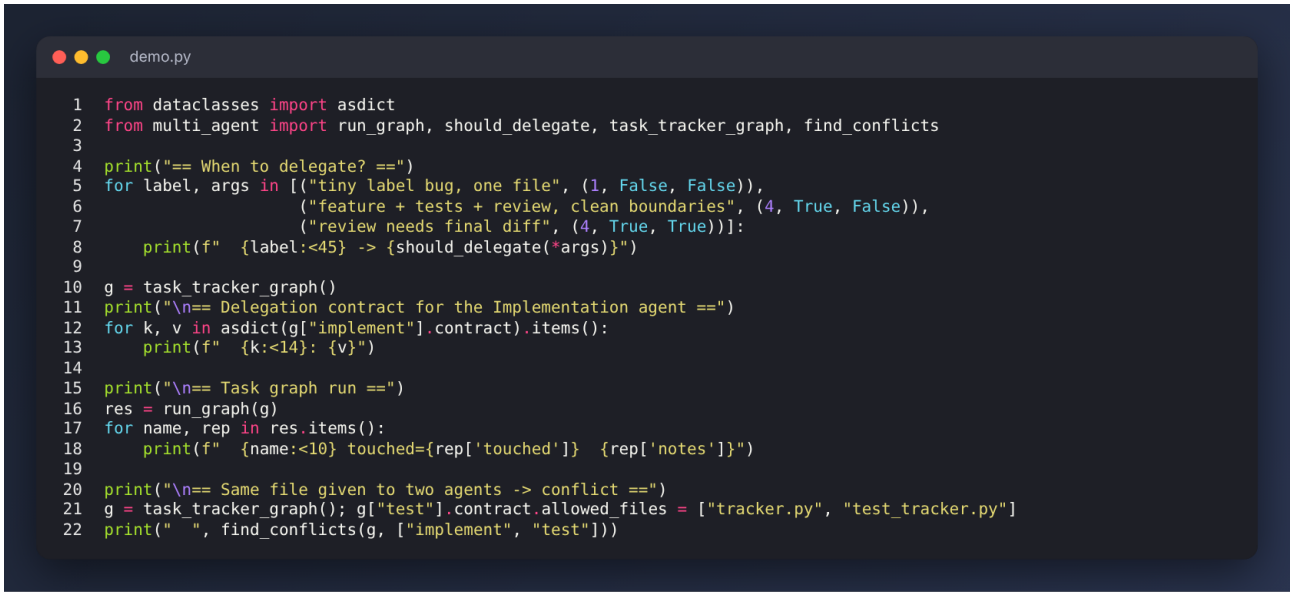


```python
def run_graph(nodes: Dict[str, Node], log: Callable[[str], None] = print) -> Dict[str, dict]:
    for n in nodes.values():
        missing = n.contract.validate()
        if missing:
            raise ValueError(f"{n.role}: contract missing {missing}")
    results: Dict[str, dict] = {}
    for lvl in levels(nodes):
        conflicts = find_conflicts(nodes, lvl)
        if conflicts:
            log(f"  conflict -> running level sequentially: {conflicts}")
            groups = [[n] for n in lvl]
        else:
            groups = [lvl]
        for g in groups:
            log(f"  level {'||'.join(g)}" + ("  (parallel)" if len(g) > 1 else ""))
            with ThreadPoolExecutor(max_workers=len(g)) as ex:
                futs = {n: ex.submit(nodes[n].agent, nodes[n].contract, results) for n in g}
                for n, f in futs.items():
                    rep = f.result()
                    enforce_scope(nodes[n], rep)
                    results[n] = rep
    return results


# ----------------------- simulated Task Tracker roles -----------------------
def coordinator(c, r):  return {"touched": [], "notes": "scope fixed: status filter only"}
def implementer(c, r):  return {"touched": ["tracker.py"], "notes": "added status predicate (diff v1)"}
def tester(c, r):       return {"touched": ["test_tracker.py"], "notes": "cases: all/open/done + negative"}
def reviewer(c, r):
    impl = r["implement"]["notes"]
    return {"touched": [], "notes": f"reviewed final diff: '{impl}'. NOTE: reviewer shares implementer assumptions ->


def task_tracker_graph() -> Dict[str, Node]:
    mk = lambda task, files, out: Contract(task, ["acceptance criteria", "TaskList + tests"], files,
                                           ["no refactor", "existing stack"], out, "focused tests", "blocked / criter
    return {
        "coordinate": Node("Coordinator", mk("define scope", [], "scope note"), coordinator),
        "implement": Node("Implementation", mk("status filter", ["tracker.py"], "diff + notes"), implementer, ["coordi
        "test": Node("Test", mk("test cases", ["test_tracker.py"], "tests"), tester, ["coordinate"]),
        "review": Node("Review", mk("review diff", [], "findings"), reviewer, ["implement", "test"]),
    }
```

Main implementation: multi_agent.py (part 2/2)



demo.py

```python
from dataclasses import asdict
from multi_agent import run_graph, should_delegate, task_tracker_graph, find_conflicts

print("== When to delegate? ==")
for label, args in [("tiny label bug, one file", (1, False, False)),
                    ("feature + tests + review, clean boundaries", (4, True, False)),
                    ("review needs final diff", (4, True, True))]:
    print(f"  {label:<45} -> {should_delegate(*args)}")

g = task_tracker_graph()
print("\n== Delegation contract for the Implementation agent ==")
for k, v in asdict(g["implement"].contract).items():
    print(f"  {k:<14}: {v}")

print("\n== Task graph run ==")
res = run_graph(g)
for name, rep in res.items():
    print(f"  {name:<10} touched={rep['touched']}  {rep['notes']}")

print("\n== Same file given to two agents -> conflict ==")
g = task_tracker_graph(); g["test"].contract.allowed_files = ["tracker.py", "test_tracker.py"]
print("  ", find_conflicts(g, ["implement", "test"]))
```

Demo script

6. Demo and test screenshots

terminal - run the tests

```
$ python -m unittest discover -v
test_agent_outside_contract_is_rejected (test_multi_agent.MultiAgentTests.test_agent_outside_contract_is_rejected) ... ok
test_cycle_detected (test_multi_agent.MultiAgentTests.test_cycle_detected) ... ok
test_delegation_table (test_multi_agent.MultiAgentTests.test_delegation_table) ... ok
test_incomplete_contract_rejected (test_multi_agent.MultiAgentTests.test_incomplete_contract_rejected) ... ok
test_levels_parallel_impl_and_test_then_review
(test_multi_agent.MultiAgentTests.test_levels_parallel_impl_and_test_then_review) ... ok
test_review_runs_after_implementation (test_multi_agent.MultiAgentTests.test_review_runs_after_implementation) ... ok
test_same_file_conflict_detected_and_serialised
(test_multi_agent.MultiAgentTests.test_same_file_conflict_detected_and_serialised) ... ok

----------------------------------------------------------------------
Ran 7 tests in 0.004s

OK
```

Test run (real output)

terminal - python demo.py

```
$ python demo.py
== When to delegate? ==
  tiny label bug, one file                    -> ONE AGENT - avoid coordination overhead
  feature + tests + review, clean boundaries  -> DELEGATE - feature + tests + review with clean boundaries
  review needs final diff                     -> SEQUENTIAL - implementation -> review

== Delegation contract for the Implementation agent ==
  task          : status filter
  inputs        : ['acceptance criteria', 'TaskList + tests']
  allowed_files : ['tracker.py']
  constraints   : ['no refactor', 'existing stack']
  output        : diff + notes
  verify        : focused tests
  stop          : blocked / criteria met

== Task graph run ==
  level coordinate
  level implement||test  (parallel)
  level review
  coordinate touched=[]  scope fixed: status filter only
  implement  touched=['tracker.py']  added status predicate (diff v1)
  test       touched=['test_tracker.py']  cases: all/open/done + negative
  review     touched=[]  reviewed final diff: 'added status predicate (diff v1)'. NOTE: reviewer shares implementer
assumptions -> not independent proof

== Same file given to two agents -> conflict ==
   ["tracker.py: both 'implement' and 'test' may edit it"]
```

Demo output (python demo.py)

7. Evidence-backed report

Change	How a Multi-Agent System Is Built
Revision tested	content hash `7537920ef876` of all .py files in this folder (re-run with the git commit SHA after you commit)
Checks run	`python -m unittest discover` and `python demo.py`
Results	**PASS** - Ran 7 tests in 0.004s
Checks not run	Real LLM agents (roles are simulated functions)
Known limitations	Parallelism uses threads on simple functions; real agents would need isolated worktrees.
Next required action	Human review, then CI on the pull request

8. Practical takeaway

Delegate only when boundaries are clear and the benefit outweighs coordination cost.